

Bloque 05_windows

Índice de módulos

- AlwaysInstallElevated
- Secuestro de DLL
- Enumeracion para privesc en Windows
- Exploits de kernel en Windows (ultimo recurso)
- Autoruns y binarios de arranque
- Privilegios SeBackup / SeRestore
- Servicios mal configurados
- Credenciales guardadas en Windows
- Impersonation de token (familia Potato)
- Bypass de UAC

AlwaysInstallElevated

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

AlwaysInstallElevated es una política de Windows que, si esta activada en las dos claves de registro (HKCU y HKLM), hace que cualquier paquete .msi se instale con privilegios de SYSTEM sin importar quien lo lance. Un atacante genera un .msi malicioso y lo instala para ejecutar código como SYSTEM.

La política existe para que usuarios sin privilegios puedan instalar software aprobado, pero abre un agujero enorme: el instalador corre como SYSTEM y el .msi puede hacer cualquier cosa. Es un vector limpio y fiable cuando esta activo. Caso del winprivesc-modelo.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En la enumeración (enum-privesc-windows). Se comprueba leyendo las dos claves de registro; WinPEAS lo marca explícitamente.

Cómo identificarla

- Ambas claves AlwaysInstallElevated valen 1 (HKCU y HKLM).
- WinPEAS resalta 'AlwaysInstallElevated set to 1'.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

reg query en ambas claves AlwaysInstallElevated (HKCU y HKLM); las dos deben ser 0x1.

Evidencia esperada: 0x1 en ambas claves. Tras instalar tu .msi, tu usuario aparece en el grupo Administrators.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.

5. Aplica una sola adaptación cuando haya evidencia de restricción.

6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
```

Objetivo y contexto: Comprueba la clave de usuario. La escalada solo funciona si tanto HKCU como HKLM tienen AlwaysInstallElevated = 1; una sola no basta.

Resultado esperado: El valor (0x1 = activado). Repite la consulta en HKLM. Si ambas son 0x1, el vector aplica.

Capa 2: prueba documentada

```
msiexec /quiet /qn /i C:\Windows\Temp\pe.msi
```

Objetivo y contexto: Instala tu .msi en silencio. Con la política activa, msiexec lo ejecuta como SYSTEM, corriendo el payload que metiste dentro (crear admin o shell).

Resultado esperado: El payload se ejecuta como SYSTEM (p. ej. tu usuario nuevo aparece en 'net localgroup administrators'). Genera el .msi con la acción deseada de antemano.

Anatomía de los payloads

La primera prueba es `reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

0x1 en ambas claves. Tras instalar tu .msi, tu usuario aparece en el grupo Administrators.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Comprueba las dos claves de registro; deben valer 1 las dos.
- Genera un .msi malicioso (crea un admin o lanza tu shell).
- Transfiere y instalalo con msiexec en modo silencioso.
- Confirma privilegios de SYSTEM/admin.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- Cualquiera de las dos claves (HKCU o HKLM) vale 0 o no existe.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Si esta política esta activa, cualquier .msi se instala como SYSTEM: generas un instalador malicioso y listo. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (always-install-elevated)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Secuestro de DLL

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Cuando un programa carga una DLL sin ruta absoluta, Windows la busca por un orden de directorios. Si un proceso privilegiado busca una DLL que no existe (o busca antes en un directorio donde puedes escribir), colocas ahí una DLL maliciosa con ese nombre y el proceso la carga y ejecuta tu código. Es el análogo Windows del path-hijacking de Linux.

El orden de búsqueda de DLLs incluye el directorio del ejecutable y otros; si la app no fija rutas y falta una DLL, el hueco es explotable. Tu DLL (con su DllMain) ejecuta el payload en el contexto del proceso, que puede ser SYSTEM.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando un servicio o app privilegiada tiene una DLL ausente o cargable desde un directorio escribible. ProcMon (filtrando NAME NOT FOUND en .dll) lo revela.

Cómo identificarla

- ProcMon muestra un proceso privilegiado con 'NAME NOT FOUND' buscando una .dll.
- El directorio donde busca la DLL es escribible por tu usuario.
- Una app de terceros que carga DLLs desde su propia carpeta escribible.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Filtra ProcMon por NAME NOT FOUND y Path ends with .dll mientras el proceso privilegiado corre.

Evidencia esperada: Una ruta de DLL no encontrada en un directorio donde puedes escribir; tras colocar tu DLL, ejecución en el contexto del proceso.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

`procmon.exe` (filtro: `Result is NAME NOT FOUND, Path ends with .dll`)

Objetivo y contexto: ProcMon monitoriza en vivo que ficheros abre cada proceso. Filtrando DLLs no encontradas ves exactamente que hueco puedes rellenar con tu DLL.

Resultado esperado: Rutas de DLLs que un proceso busca sin éxito. Si alguna está en un directorio escribible por ti y el proceso es privilegiado, ese es tu vector.

Capa 2: prueba documentada

`x86_64-w64-mingw32-gcc pe.c -shared -o legit.dll`

Objetivo y contexto: Compila (desde Kali) una DLL cuyo `DllMain` ejecuta tu payload al cargarse. El nombre debe coincidir con la DLL que el proceso busca.

Resultado esperado: El fichero `legit.dll`. Colocalo en el directorio donde el proceso la busca; al reiniciarse/cargarla, ejecuta tu código en su contexto (idealmente SYSTEM).

Anatomía de los payloads

La primera prueba es `procmon.exe` (filtro: `Result is NAME NOT FOUND, Path ends with .dll`). Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Una ruta de DLL no encontrada en un directorio donde puedes escribir; tras colocar tu DLL, ejecución en el contexto del proceso.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Con ProcMon, caza DLLs que un proceso privilegiado busca y no encuentra.
- Confirma que puedes escribir en un directorio del orden de búsqueda.
- Compila una DLL con ese nombre cuyo `DllMain` lance tu payload.
- Colocala y dispara/espera la carga; confirma la ejecución privilegiada.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- Ningún proceso privilegiado busca una DLL en un directorio escribible por ti.

- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Una app privilegiada que busca una DLL en un directorio escribible carga la tuya: es el PATH hijacking de Windows. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.

4. Interpretar señal antes de escalar.

5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(dll-hijacking\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Enumeracion para privesc en Windows

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Enumerar para privesc en Windows es recorrer sus vectores típicos en orden: privilegios de token, grupos, servicios mal configurados, tareas programadas, credenciales guardadas y parches faltantes. WinPEAS lo automatiza; entender cada hallazgo es lo que lo convierte en escalada. Paso previo del winprivesc-modelo.

Windows esconde muchos vectores en configuración (servicios, registro, permisos NTFS) que a mano son tediosos. WinPEAS y las consultas manuales garantizan cobertura, pero dan falsos positivos: hay que validar cada pista.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Nada más conseguir una shell en Windows. Corre primero whoami /priv y whoami /groups, luego WinPEAS para lo exhaustivo.

Cómo identificarla

- whoami /priv revela un privilegio potente (SeImpersonate, SeBackup, SeDebug).
- WinPEAS resalta servicios modificables o rutas sin comillas.
- Credenciales en ficheros de despliegue o en el gestor de credenciales.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

whoami /priv y whoami /groups primero; después winPEAS para el resto.

Evidencia esperada: Un privilegio de token Enabled, o un hallazgo resaltado en winPEAS (servicio, credencial, AlwaysInstallElevated).

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

`whoami /priv`

Objetivo y contexto: Lista tus privilegios de token, el vector mas rapido en Windows. Varios (Selpersonate, SeBackup, SeRestore, SeDebug) llevan a SYSTEM casi directo.

Resultado esperado: Privilegios y si estan Enabled. Selpersonate -> token-impersonation. SeBackup/SeRestore -> sebackup-serestore. Anota los que esten habilitados.

Capa 2: prueba documentada

`systeminfo`

Objetivo y contexto: Muestra version de Windows y hotfixes instalados. Comparar los parches con los CVE conocidos revela si hay un exploit de kernel aplicable.

Resultado esperado: OS version, build y lista de KB (parches). Pocos KB en un Windows viejo -> candidato a exploit de kernel; pasalo por Watson/wesng.

Anatomía de los payloads

La primera prueba es `whoami /priv`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un privilegio de token Enabled, o un hallazgo resaltado en winPEAS (servicio, credencial, AlwaysInstallElevated).

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Checks rapidos: `whoami /priv`, `whoami /groups`, `systeminfo` (parches).
- Lanza WinPEAS y lee lo resaltado (servicios, credenciales, AlwaysInstallElevated).
- Prioriza por rentabilidad: token (token-impersonation) y servicios (service-misconfig-windows) primero.
- Deja los exploits de kernel (kernel-exploits-windows) para el final.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- Ya recorriste privilegios de token, servicios y credenciales sin hallazgos.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.

- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Igual que en Linux pero con otros checks: privilegios de token, servicios, credenciales guardadas y parches faltantes. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (enum-privesc-windows)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Exploits de kernel en Windows (ultimo recurso)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Un exploit de kernel de Windows aprovecha un fallo en el núcleo o en un driver para ejecutar código como SYSTEM desde un usuario normal. Aplica cuando faltan parches concretos (MS16-032, PrintNightmare, etc.). Se identifican comparando los hotfixes instalados con los CVE conocidos.

Como en Linux, se dejan para el FINAL: pueden inestabilizar o tumbar la máquina, y casi siempre hay un vector de config más limpio. Solo cuando la enumeración (enum-privesc-windows) no dio token, servicio ni credencial se recurre a esto.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando ningún vector de config funciona y systeminfo revela un Windows con pocos parches. Watson/wesng cruzan los hotfixes con exploits conocidos.

Cómo identificarla

- systeminfo muestra pocos hotfixes o un build viejo.
- Watson/wesng sugieren exploits aplicables a esa build.
- Servicios de impresión vulnerables (PrintNightmare) presentes.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Pasa systeminfo por wesng/Watson y compara con los KB instalados.

Evidencia esperada: Un CVE de Elevation of Privilege sin el KB correspondiente instalado.

Construcción progresiva del payload

1. Reproduce una entrada válida.

2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
systeminfo > sysinfo.txt
```

Objetivo y contexto: Captura version, build y hotfixes. Es la entrada para las herramientas que sugieren que exploit de kernel aplica a esta maquina.

Resultado esperado: OS version/build y lista de KB. Guardalo y pasalo por wesng: te dara CVEs sin parchear con exploit publico.

Capa 2: prueba documentada

```
python wesng.py sysinfo.txt
```

Objetivo y contexto: Windows Exploit Suggester cruza tu systeminfo con la base de vulnerabilidades y lista exploits candidatos ordenados por relevancia.

Resultado esperado: Lista de CVE/MS con enlace al exploit. Prioriza los de 'Elevation of Privilege' con PoC; leelo antes de ejecutar (exploitar-cve).

Anatomía de los payloads

La primera prueba es `systeminfo > sysinfo.txt`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un CVE de Elevation of Privilege sin el KB correspondiente instalado.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Saca version y parches con systeminfo.
- Pasa la salida por wesng/Watson para candidatos.
- Elige uno, leelo (exploitar-cve) y consigue el binario compilado fiable.
- Ejecuta con cuidado; confirma SYSTEM. Si falla o cuelga, vuelve a los vectores de config.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El sistema esta al día de parches para los CVEs conocidos.
- Quedan vectores de configuracion sin probar (token, servicios, credenciales): prueba esos primero.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Si faltan parches, un exploit local eleva a SYSTEM; deja esto para el final por inestabilidad. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y precondiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (kernel-exploits-windows)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Autoruns y binarios de arranque

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Windows ejecuta programas al inicio o al login desde varias claves de registro (Run, RunOnce) y carpetas de startup, a veces con privilegios altos. Si el binario referenciado es escribible por tu usuario, o la clave de registro es modificable, colocas tu ejecutable y corre con los privilegios del que dispara el autorun. Variante del winprivesc-modelo.

El software de terceros registra autoruns sin proteger bien los permisos del binario o de la clave. Si un admin inicia sesión (o el arranque es como SYSTEM), tu binario reemplazado se ejecuta con ese contexto.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En la enumeración (enum-privesc-windows). WinPEAS y autorunsc listan los autoruns y marcan los que tienen permisos débiles.

Cómo identificarla

- Un binario referenciado en una clave Run es escribible por tu usuario (accesschk).
- Una clave de registro de autorun modificable por ti.
- autorunsc marca un autorun con permisos flojos.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

accesschk sobre el binario referenciado por cada entrada de Run/RunOnce.

Evidencia esperada: Un permiso de escritura (FILE_WRITE_DATA/W) para tu usuario o grupo sobre el .exe de autorun.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
reg query HKLM\Software\Microsoft\Windows\CurrentVersion\Run
```

Objetivo y contexto: Lista los programas que arrancan con el sistema desde la clave Run de maquina. Son candidatos si el binario o la clave son escribibles.

Resultado esperado: Nombres y rutas de los binarios de autorun. Para cada uno, comprueba con accesschk si puedes escribir el .exe o modificar la clave.

Capa 2: prueba documentada

```
accesschk.exe /accepteula -wvu "C:\ruta\autorun.exe"
```

Objetivo y contexto: Comprueba si tu usuario tiene permiso de escritura sobre el binario de autorun. Esa escritura es lo que te deja reemplazarlo por tu payload.

Resultado esperado: Permisos por usuario/grupo. Un FILE_WRITE_DATA/W para tu usuario o un grupo tuyo = puedes reemplazar el .exe. Si no, prueba modificar la clave de registro.

Anatomía de los payloads

La primera prueba es `reg query HKLM\Software\Microsoft\Windows\CurrentVersion\Run`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un permiso de escritura (FILE_WRITE_DATA/W) para tu usuario o grupo sobre el .exe de autorun.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Enumera autoruns (registro Run/RunOnce y carpetas startup).
- Comprueba permisos del binario y de la clave (accesschk).
- Reemplaza el binario escribible por tu payload (o apunta la clave al tuyo).
- Espera el proximo login/arranque del usuario privilegiado y recoge la ejecucion.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- Ningun autorun referencia un binario escribible por tu usuario, y las claves de registro no son modificables.

- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Si un programa que arranca automáticamente con privilegios apunta a un binario que puedes escribir, lo reemplazas. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.

4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (registry-autoruns)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Privilegios SeBackup / SeRestore

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

SeBackupPrivilege permite leer cualquier fichero ignorando sus ACL (pensado para software de backup), y SeRestorePrivilege permite escribir cualquiera. Con SeBackup puedes copiar los hives SAM y SYSTEM del registro, llevartelos y extraer los hashes NTLM locales, incluido el del administrador. Caso concreto del winprivesc-modelo via privilegios de token.

El privilegio existe para que las herramientas de copia lean ficheros bloqueados, pero no distingue backup legítimo de robo: si lo tienes, saltas todos los permisos NTFS. El SAM contiene los hashes locales; con ellos, ntlm-pth te da administrator sin crackear.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando whoami /priv (enum-privesc-windows) muestra SeBackupPrivilege (habitual en el grupo Backup Operators).

Cómo identificarla

- whoami /priv lista SeBackupPrivilege / SeRestorePrivilege habilitado.
- Pertenece al grupo Backup Operators.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

reg save de HKLM\SAM y HKLM\SYSTEM; si funciona sin error de acceso, el privilegio esta activo.

Evidencia esperada: Dos ficheros .hive generados sin error; tras secretdump, hashes NTLM locales incluido administrator.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
reg save HKLM\SAM sam.hive && reg save HKLM\SYSTEM system.hive
```

Objetivo y contexto: Con SeBackup puedes exportar los hives del registro que contienen los hashes locales (SAM) y la clave para descifrarlos (SYSTEM), saltando el bloqueo habitual.

Resultado esperado: Dos ficheros .hive. Baja ambos a tu Kali; el siguiente paso es extraer los hashes con impacket-secretsdump usando SAM + SYSTEM.

Capa 2: prueba documentada

```
impacket-secretsdump -sam sam.hive -system system.hive LOCAL
```

Objetivo y contexto: Extrae los hashes NTLM locales combinando SAM y SYSTEM. Es el paso que convierte los hives robados en credenciales usables.

Resultado esperado: Líneas usuario:rid:lm:nt. El NT hash de Administrator -> ntlm-pth con evil-winrm/psexec para shell como admin sin crackear.

Anatomía de los payloads

La primera prueba es `reg save HKLM\SAM sam.hive && reg save HKLM\SYSTEM system.hive`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Dos ficheros .hive generados sin error; tras secretsdump, hashes NTLM locales incluido administrator.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Confirma SeBackup en whoami /priv.
- Copia los hives SAM y SYSTEM (via reg save o robocopy /b).
- Transfiérellos a tu Kali y extrae los hashes con secretsdump.
- Usa el hash NTLM de administrator con ntlm-pth para shell como admin.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- whoami /priv no muestra esos privilegios habilitados.
- Ejecutar la prueba sin adaptar variables ni versión.

- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Estos privilegios saltan las ACL para leer/escribir cualquier fichero: volcas el SAM y sacas el hash del administrador. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.

5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (sebackup-serestore)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Servicios mal configurados

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Los servicios de Windows suelen correr como SYSTEM. Si tu usuario puede modificar la configuración del servicio (binPath), reemplazar su ejecutable, o si la ruta del binario no está entre comillas y hay un directorio escribible en medio, puedes hacer que el servicio ejecute TU binario como SYSTEM. Es el winprivesc-modelo aplicado a servicios.

Instaladores descuidados dejan servicios con permisos NTFS flojos en el .exe o permiten a usuarios normales reconfigurar el servicio. Y una ruta como C:\Program Files\My App\svc.exe sin comillas hace que Windows pruebe C:\Program.exe primero: si puedes escribir ahí, secuestras el arranque.

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                    -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En la enumeración (enum-privesc-windows). WinPEAS marca servicios modificables, binarios escribibles y unquoted paths.

Cómo identificarla

- accesschk o WinPEAS marca SERVICE_CHANGE_CONFIG o WRITE sobre un servicio.
- El .exe del servicio está en una carpeta escribible por tu usuario.
- Una ruta de servicio con espacios y sin comillas, con un dir intermedio escribible.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

sc qc <servicio> para ver BINARY_PATH_NAME y SERVICE_START_NAME; cruzalo con accesschk.

Evidencia esperada: Una ruta sin comillas con directorio intermedio escribible, o permiso de reconfigurar el servicio (SERVICE_CHANGE_CONFIG).

Construcción progresiva del payload

1. Reproduce una entrada válida.

2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
sc qc <servicio>
```

Objetivo y contexto: Muestra la configuración del servicio: su BINARY_PATH_NAME y la cuenta con la que corre. Ahí ves si la ruta está sin comillas y si corre como LocalSystem.

Resultado esperado: BINARY_PATH_NAME (mira comillas y espacios) y SERVICE_START_NAME (LocalSystem = SYSTEM). Combinar con accesschk revela si puedes modificarlo.

Capa 2: prueba documentada

```
sc config <servicio> binPath= "C:\Windows\Temp\pe.exe" && sc start <servicio>
```

Objetivo y contexto: Si tienes SERVICE_CHANGE_CONFIG, reescribes que ejecuta el servicio y lo arrancas: tu binario corra como SYSTEM. Es el abuso más directo.

Resultado esperado: El servicio ejecuta tu pe.exe como SYSTEM (crea un usuario admin o lanza tu shell). Si no puedes cambiar config, prueba reemplazar el .exe o el unquoted path.

Anatomía de los payloads

La primera prueba es `sc qc <servicio>`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Una ruta sin comillas con directorio intermedio escribible, o permiso de reconfigurar el servicio (SERVICE_CHANGE_CONFIG).

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Enumera servicios y sus permisos (accesschk / sc qc).
- Según el fallo: reconfigura binPath, reemplaza el .exe, o coloca un exe en la ruta sin comillas.
- Reinicia el servicio (o espera) para que ejecute tu payload como SYSTEM.
- Confirma con whoami que eres nt authority\system.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- Todos los servicios corren con permisos correctos y rutas entre comillas.
- No tienes SERVICE_CHANGE_CONFIG ni escritura sobre ningún binario de servicio.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Un servicio que corre como SYSTEM con permisos flojos: si modificas su binario, su binPath o una ruta sin comillas, ejecutas como SYSTEM. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y precondiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (service-misconfig-windows)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Credenciales guardadas en Windows

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Muchas escaladas en Windows no son un exploit, sino encontrar una credencial reutilizable. Windows guarda passwords en ficheros de despliegue (unattend.xml, sysprep), en el gestor de credenciales (cmdkey), en el registro (Winlogon autologon, PuTTY), y en backups del SAM/SYSTEM que se pueden volcar. Cada credencial alimenta cracking o reuse directo.

Los administradores automatizan despliegues y dejan credenciales en claro o cifradas de forma reversible. Y el propio Windows cachea secretos para comodidad. Buscar sistemáticamente es de lo mas rentable, igual que en Linux.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En la enumeracion (enum-privesc-windows), en paralelo a los vectores de servicio/token. WinPEAS busca la mayoría automáticamente.

Cómo identificarla

- Ficheros unattend.xml / sysprep.xml / Autounattend.xml en el disco.
- cmdkey /list muestra credenciales cacheadas (runas /savecred).
- Autologon en el registro con DefaultPassword, o hives SAM/SYSTEM accesibles.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

findstr /si password sobre configs y reg query del autologon en Winlogon.

Evidencia esperada: Una password en claro en un fichero de config, o DefaultPassword en el registro.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon" /v DefaultPassword
```

Objetivo y contexto: Si hay autologon configurado, la password del usuario queda en claro en el registro. Es un clasico de rooms Windows.

Resultado esperado: DefaultUserName y DefaultPassword en claro si existe. Esa credencial -> pruebala con runas o para RDP/WinRM; a menudo es de un usuario mas privilegiado.

Capa 2: prueba documentada

```
findstr /si password *.xml *.ini *.txt *.config
```

Objetivo y contexto: Peina ficheros de configuracion y despliegue buscando 'password'. /s recursivo, /i sin distinguir mayusculas. Barato y muy rentable.

Resultado esperado: Lineas con credenciales y su fichero. unattend.xml suele traer la password del admin local (a veces en base64: decodificala).

Anatomía de los payloads

La primera prueba es `reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon" /v DefaultPassword`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Una password en claro en un fichero de config, o DefaultPassword en el registro.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Busca ficheros de despliegue con credenciales en claro o base64.
- Lista credenciales cacheadas con cmdkey; usa runas /savecred si hay.
- Revisa el registro (autologon, claves de apps) y volca SAM/SYSTEM si puedes.
- Cada credencial -> pruebala (reuse) o crackeala (cracking); un hash NTLM sirve para ntlm-pth.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- No hay ficheros de despliegue (unattend.xml), autologon ni credenciales cacheadas visibles.
- Ejecutar la prueba sin adaptar variables ni versión.

- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Windows deja credenciales por todas partes: ficheros de despliegue, gestor de credenciales, registro y backups del SAM. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.

5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (stored-credentials-windows)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Impersonation de token (familia Potato)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

SelmpersonatePrivilege permite a un proceso actuar con el token de seguridad de otro cliente que se conecte a él. Los exploits de la familia Potato (JuicyPotato, PrintSpoofer, GodPotato) engañan a un servicio privilegiado para que se autentique contra ellos y así capturan y suplantan el token de SYSTEM, obteniendo una shell SYSTEM.

Las cuentas de servicio (IIS, MSSQL) suelen tener Selmpersonate por diseño para funcionar, pero ese mismo privilegio permite robar el token de SYSTEM. Es tan común que ver Selmpersonate en whoami /priv casi garantiza la escalada. Es un caso concreto del winprivesc-modelo.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En cuanto whoami /priv muestre SelmpersonatePrivilege o SeAssignPrimaryTokenPrivilege como Enabled, típico tras comprometer una web (IIS) o un servicio en Windows.

Cómo identificarla

- SelmpersonatePrivilege = Enabled en whoami /priv.
- Tu usuario es un service account (iis apppool\..., mssql, local service).
- PrintSpoofer o GodPotato devuelven un token de SYSTEM al ejecutarlos.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Ejecuta PrintSpoofer -i -c whoami y comprueba el usuario devuelto.

Evidencia esperada: 'nt authority\system' en la salida de whoami dentro del proceso lanzado.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
PrintSpoofer64.exe -i -c cmd
```

Objetivo y contexto: Abusa de Selmpersonate forzando al spooler a autenticarse contra el exploit y suplantando su token SYSTEM. -i abre una consola interactiva como SYSTEM.

Resultado esperado: Un cmd nuevo donde whoami dice 'nt authority\system'. Si falla por version/parcheo del spooler, prueba GodPotato o JuicyPotato segun el Windows.

Anatomía de los payloads

La primera prueba es `PrintSpoofer64.exe -i -c cmd`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

'nt authority\system' en la salida de whoami dentro del proceso lanzado.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Confirma Selmpersonate con whoami /priv.
- Sube el binario adecuado (PrintSpoofer/GodPotato segun la version de Windows).
- Ejecutalo pidiendo que lance un comando como SYSTEM (whoami, o tu shell).
- Confirma nt authority\system; con hashes del SAM puedes ademas moverte via ntlm-pth.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- whoami /priv no muestra Selmpersonate ni SeAssignPrimaryToken como Enabled.
- El spooler/servicio objetivo esta parcheado contra todas las variantes de Potato conocidas.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

El privilegio Selmpersonate deja suplantar el token de SYSTEM: es la escalada mas comun en cuentas de servicio Windows. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (token-impersonation)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Bypass de UAC

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

El Control de Cuentas de Usuario (UAC) hace que, aunque tu usuario sea administrador, ejecutes con un token de integridad media hasta que confirmas la elevación. Un bypass de UAC aprovecha binarios de Windows que auto-elevan sin pedir consentimiento para lanzar un proceso con integridad alta (admin completo) sin el prompt.

Ciertos ejecutables firmados de Microsoft se auto-elevan y leen configuración del registro de usuario; si secuestras esa clave (fodhelper, eventvwr), Windows lanza tu comando elevado. No es cruzar de usuario a admin, sino de 'admin a medias' a 'admin completo'.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando `whoami /groups` muestra que estás en Administrators pero con integridad media (no puedes tocar recursos protegidos aun). No aplica si ya eres SYSTEM o integridad alta.

Cómo identificarla

- Estás en el grupo Administrators pero el nivel de integridad es Medium.
- Operaciones de admin fallan con acceso denegado pese a ser admin.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

`whoami /groups` | `findstr Label` para ver tu nivel de integridad actual.

Evidencia esperada: 'Mandatory Label\Medium'; tras el bypass, un proceso con integridad High ejecutando tu comando.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
whoami /groups | findstr /i "Label"
```

Objetivo y contexto: Muestra tu nivel de integridad (Mandatory Label). Distingue si necesitas bypass de UAC (Medium) o si ya tienes admin completo (High).

Resultado esperado: 'Mandatory Label\Medium' = necesitas UAC bypass. 'High' = ya elevado, no hace falta. Junto con el grupo Administrators define si aplica.

Capa 2: prueba documentada

```
reg add HKCU\Software\Classes\ms-settings\Shell\Open\command /d "cmd.exe" /f
```

Objetivo y contexto: Prepara el secuestro de fodhelper: al ejecutarse (auto-elevado), lee esta clave de usuario y lanza tu comando con integridad alta, sin prompt de UAC.

Resultado esperado: Tras esto, ejecutar fodhelper.exe abre un cmd elevado (integridad High). Ajusta la tecnica/clave a la build de Windows; limpia la clave despues.

Anatomía de los payloads

La primera prueba es `whoami /groups | findstr /i "Label"`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

'Mandatory Label\Medium'; tras el bypass, un proceso con integridad High ejecutando tu comando.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Confirma que eres admin con integridad media (`whoami /groups`).
- Elige una tecnica segun la build (`fodhelper`, `eventvwr`, `computerdefaults`).
- Secuestra la clave de registro correspondiente para que apunte a tu comando.
- Dispara el binario auto-elevado; tu comando corre con integridad alta.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- Tu usuario no esta en Administrators (esto no es una escalada de usuario a admin, solo de admin-medio a admin-alto).
- Ya tienes integridad High/System.

- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Ya eres admin pero con token limitado; un bypass de UAC eleva a integridad alta sin el prompt de consentimiento. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.

4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (uac-bypass)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.